COS 214 – TEAM OBSIDIAN

Team Mates:

- Ngeletshedzo Mutwanamba (Team Leader) – u25039769
- Amira Ajanaku – u25111699
- Nare Maela - u24607917
- Nontokozo Madela - u25677404
- Manqoba Mashego – u24589137
- Giselle Mpye - u25622103
- Shepherd Nzimande - u25066120

Github Repo:

https://github.com/u25622103-droid/COS214-PROJ

Task 1: Research Findings on Workflow Management Systems

What is a workflow? A workflow is basically a business process that has been automated, where documents, information or tasks get passed between people following a set of rules. A workflow management system (WfMS) is the software that sets these workflows up and runs them, and the workflow engine is the part that does the actual running. Business process management (BPM) is the bigger picture, because it also covers analysing processes and how work is organised, not just automating it (Workflow Management Coalition, 1994).

Definition vs instance. The definition is the blueprint: the steps in a process and the rules for moving between them. An instance is one running copy of that blueprint. Lots of instances can run at the same time from one model, and each one deals with its own case, so each needs its own state and data (Workflow Management Coalition, 1994) (van der Aalst, 2013).

Activities and sub-processes. An activity is one step in the process, and when a person must do it, a work item shows up on their worklist. BPMN 2.0 splits tasks into user tasks (a human does it) and service tasks (a system does it). It also has sub-processes, which are activities with their own flow inside, and call activities, which reuse a process that is defined somewhere else (Object Management Group, 2011).

Routing. The workflow patterns research lists the basic ways to route work: sequence, parallel split (AND), synchronisation, exclusive choice (XOR), simple merge, multi-choice and loops. Different products support different ones, so it makes sense to keep routing flexible and not hard-code it (van der Aalst, et al., 2003).

Participants and assigning work. Engines look after worklists and who is signed in. The BPM survey describes people using roles and organisational units, including rules like the four-eyes principle, so the same person cannot do two dependent tasks. Work can reach people in two ways: the system pushes it to a worker, or the worker pulls it from the system (Workflow Management Coalition, 1994; van der Aalst, 2013; Russell et al., 2005) (Russell, et al., 2005).

Events, failures and external systems. In BPMN, boundary events sit on the edge of an activity and react while it is running. There are timer, message, error and escalation events. Compensation can undo activities that already finished if something goes wrong. The WfMC model also has separate interfaces for calling outside applications and talking to other workflow systems (Object Management Group, 2011).

History and changing definitions. Everything that happens gets saved in event logs, where each event points to an activity and a case, and process mining uses these logs to find out how processes really run. The WfMC treats audit as part of monitoring. To see how a real product deals with a definition changing while work is still running, we looked at the documentation of Camunda, a company that makes a workflow engine. In Camunda, a changed definition is saved as a new version. Running instances stay on the old version, new ones use the latest, and teams can choose to move running instances over without losing their history (van der Aalst, 2013) (Workflow Management Coalition, 1994) (Camunda, n.d).

What this means for TIVIDY. Our takeaway is that the engine should only know generic things like activities, instances, states, participants and events. Anything that changes between processes (the structure and versions, how work is grouped, routing, assignment and approval rules, events and integrations, and reports and rollback) should be able to change without touching the rest.

Task 2: Defining the scenario and system

In a meal plan order-to-delivery shopping system that offers custom meal packages for customers; Breakfast, Lunch and Diner. It manages customer orders from payment and coupon discounts, through kitchen cooking and packaging and finally courier handoff and delivery. The goal is to deliver the meal package to the customer successfully while keeping an accurate record of every step.

Workflow Definition (The Blueprint): A master checklist that says what steps must happen in what order and under what rules. When setting up a new meal plan, TIVIDY constructs the master template step by step. For example: It puts together the recipe: First validate coupon -> then prepare meals -> then package food -> then send via regional courier. (**Builder**)
Workflow instance (The active order): This is one order running through the system for a specific customer. TIVIDY creates brand new instances for a customer, and creates the exact task objects she needs (e.g. Breakfast or Lunch). A customer's order moves through cooking, packaging, and delivery independently. If the customer changes their address or cancels their order, it only affects their instance. Other customers' orders keep running smoothly without any bugs or interruptions. (**Factory Method**)

People and the Roles involved:

- **Customers:** choose meal package and place order, then applies discount coupons if any.
- **Order engine(TIVIDY):** Manages the process steps to complete order.
- **Kitchen staff:** Cooks food and packages meals
- **3 Delivery Services:** Three different third-party courier companies that handle final delivery in different regions. TIVIDY connects their different systems using special bridges (**Adapter**).
- **Branch Manager:** A manager who resolves issues if food items run out or delivery fails (**Chain of Responsibility**).

Main Work Items and Packages

- **Meal Packages:** Orders can be single meals (Breakfast, Lunch, Dinner) or a full day bundle that groups them together (**Composite**).
- **Discounts**: Dynamic coupon rules wrap around the base order cost to apply savings based on the price of the order (**Decorator**).
- **Packaging:** Kitchen workers pack the meals once preparation is complete.

Decisions, Rules and Steps

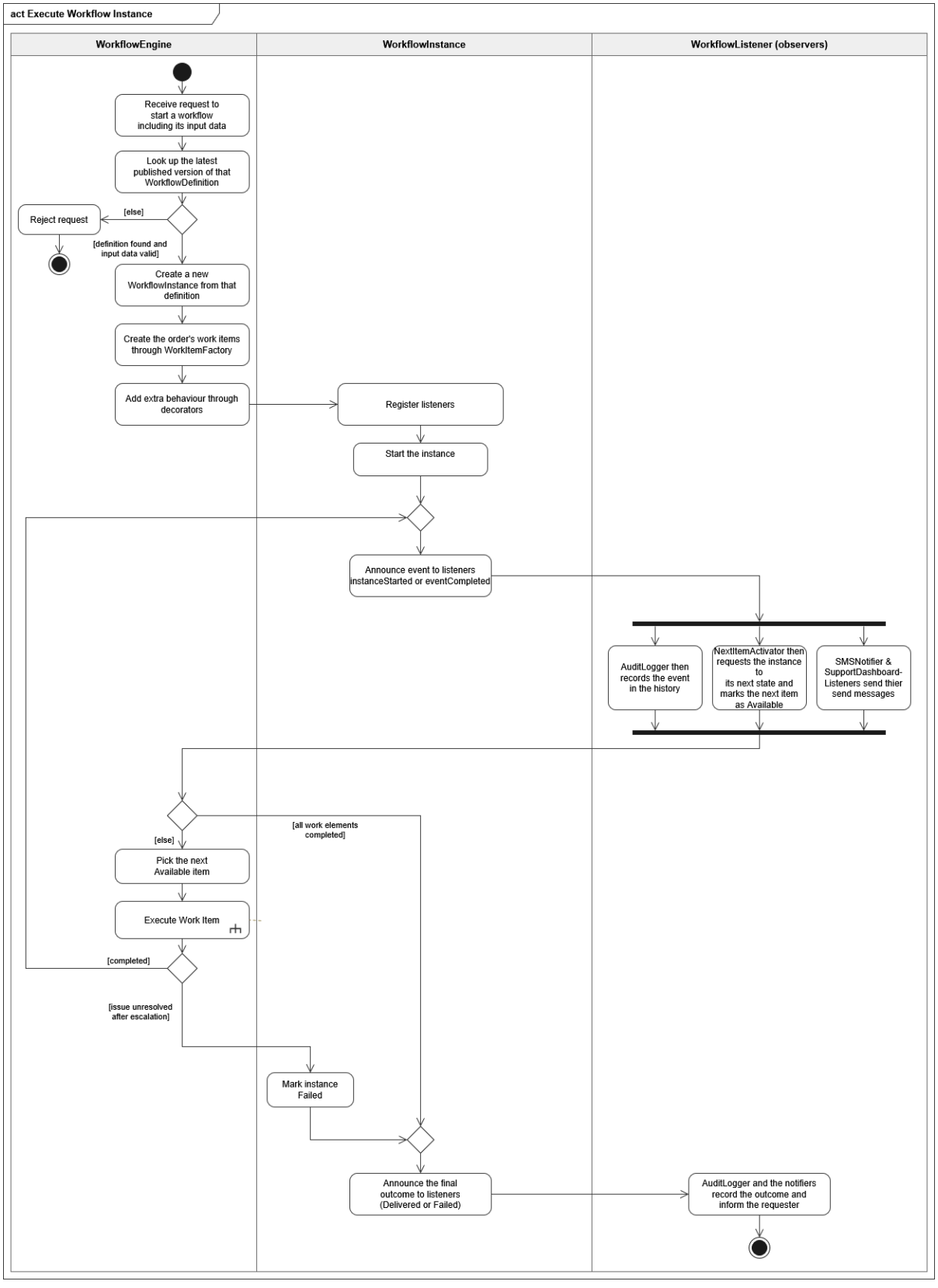
- **Status Updates:** An order moves through clear statuses (Created -> Preparing -> Packaging -> In transit -> Delivered), and only allows valid actions each step (**State**).
- **Dependencies:** Packing can only start after all selected meals in a bundle are cooked and ready.
- **Delivery Routing:** TIVIDY automatically picks 1 of the 3 delivery companies based on the customer's delivery area (**Strategy**).
- **Live Alerts:** When kitchen staff finishes packaging, the order alerts the customer support and sends an SMS to the user automatically (**Observer**).

Fixing Problems and Undo Feature

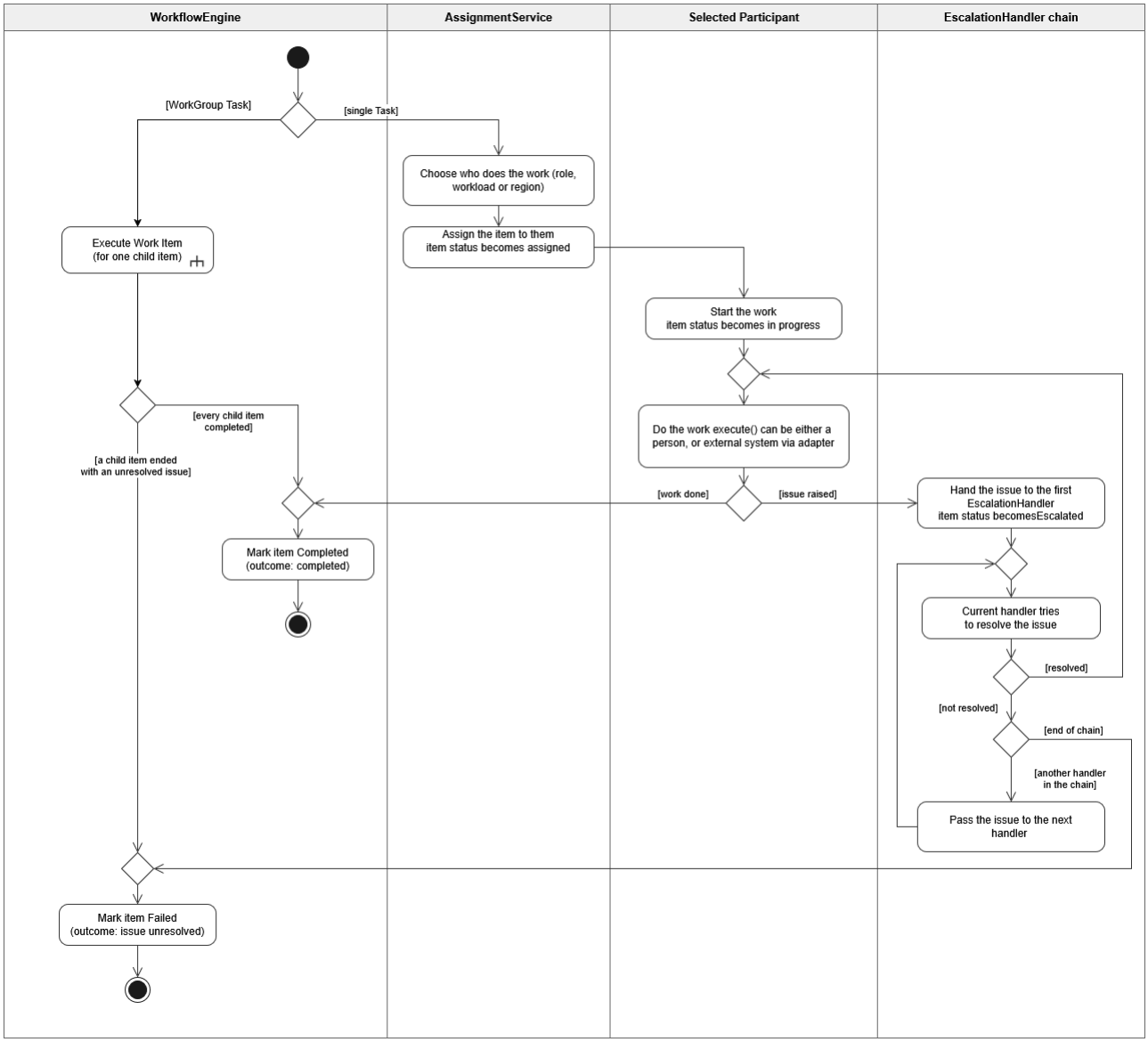
- **Out of stock issues:** If ingredients run out, the issue gets passed up the chain from Kitchen staff to Supervisor to Manager until its solved (**Chain of Responsibility**).
- **Cancelling or RollBack:** If a user cancels an order or changes a coupon, TIVIDY takes a quick snapshot of the previous state to roll back cleanly without ruining history records (**Memento**).

Task 3: UML Activity Diagrams

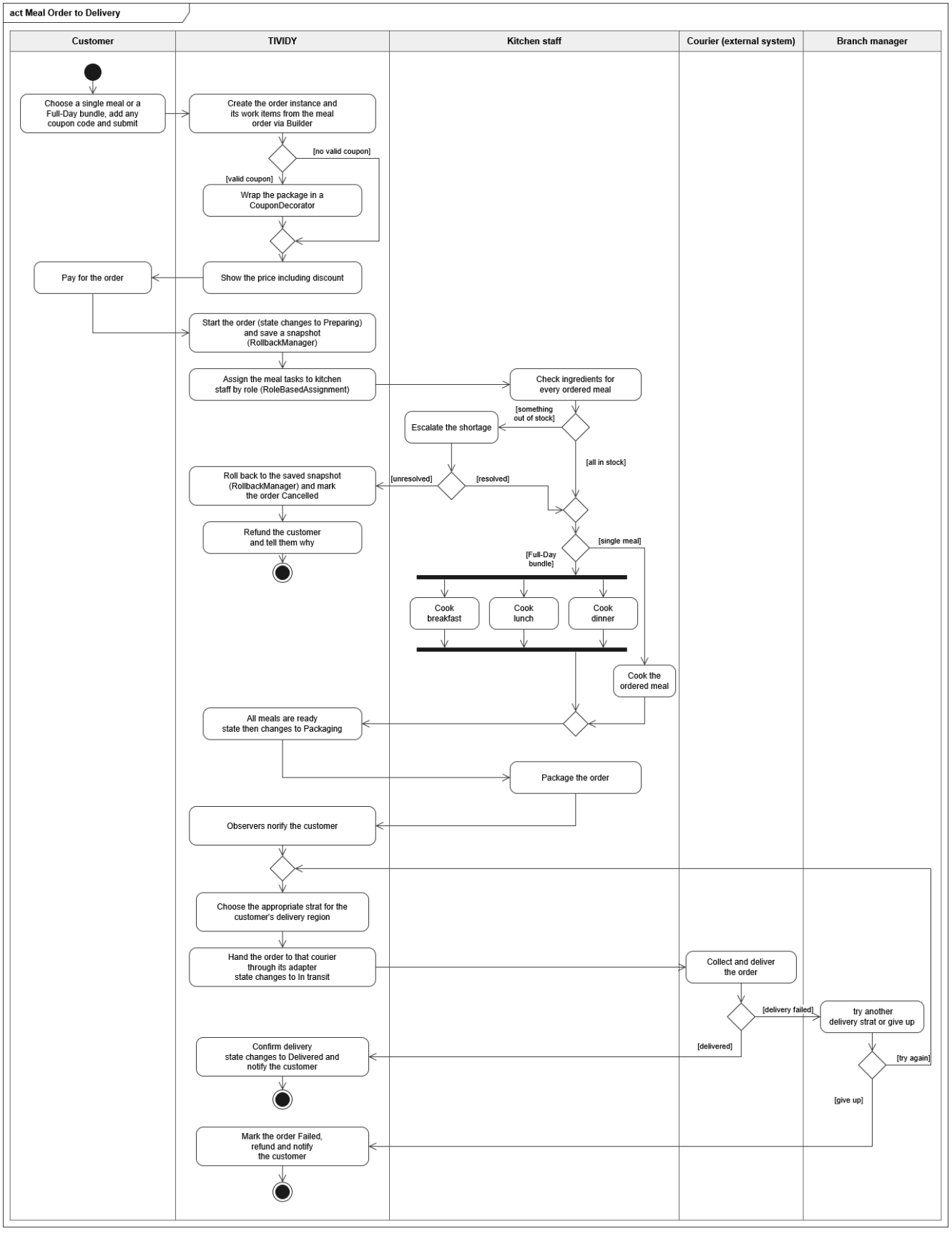
1.



2.


WorkflowEngine
AssignmentService
Selected Participant
EscalationHandler chain
[WorkGroup Task]
[single Task]
Choose who does the work (role, workload or region)
Execute Work Item (for one child item)
Assign the item to them item status becomes assigned
Start the work item status becomes in progress
[every child item completed]
[a child item ended with an unresolved issue]
Do the work execute() can be either a person, or external system via adapter
[work done]
[issue raised]
Hand the issue to the first EscalationHandler item status becomesEscalated
Mark item Completed (outcome: completed)
Current handler tries to resolve the issue
[resolved]
[not resolved]
[end of chain]
[another handler in the chain]
Pass the issue to the next handler
Mark item Failed (outcome: issue unresolved)

3.


act Meal Order to Delivery
Customer
TIVIDY
Kitchen staff
Courier (external system)
Branch manager
Choose a single meal or a Full-Day bundle, add any coupon code and submit
Create the order instance and its work items from the meal order via Builder
[no valid coupon]
[valid coupon]
Wrap the package in a CouponDecorator
Pay for the order
Show the price including discount
Start the order (state changes to Preparing) and save a snapshot (RollbackManager)
Assign the meal tasks to kitchen staff by role (RoleBasedAssignment)
Check ingredients for every ordered meal
[something out of stock]
Escalate the shortage
[all in stock]
[unresolved]
[resolved]
Roll back to the saved snapshot (RollbackManager) and mark the order Cancelled
Refund the customer and tell them why
[single meal]
[Full-Day bundle]
Cook breakfast
Cook lunch
Cook dinner
Cook the ordered meal
All meals are ready state then changes to Packaging
Package the order
Observers norify the customer
Choose the appropriate strat for the customer's delivery region
Hand the order to that courier through its adapter state changes to In transit
Collect and deliver the order
[delivery failed]
try another delivery strat or give up
[delivered]
[try again]
[give up]
Confirm delivery state changes to Delivered and notify the customer
Mark the order Failed, refund and notify the customer

Task 4: Design Patterns

1. Builder

Problem: A meal-plan definition is assembled in several steps (validate coupon → prepare meals → package → courier delivery), with routing rules and an escalation chain attached. It must be complete and validated before any customer order runs from it, and it must not change under running orders.

Participants:

GoF Participants	Class Names
Builder	WorkflowDefinitionBuilder
ConcreteBuilder	MealOrderDefinitionBuilder
Director	WorkflowDesigner
Product	WorkflowDefinition

Collaboration: The WorkflowDesigner directs the WorkflowDefinitionBuilder step by step, telling it which stages, work items, assignment rules and escalation chain to include. MealOrderDefinitionBuilder records each step into the definition under construction. Once the designer finishes, the builder checks that the definition is complete and hands back a versioned WorkflowDefinition.

Variation: The steps, their order and the rules attached to them (assignment strategy, escalation chain) differ for every kind of process. A new version of the definition is produced whenever any of them changes.

Why is it appropriate: The Builder is the construction side of the definition/instance split and the only place where a definition is created. By the time a WorkflowDefinition reaches the engine it is complete, validated and read-only, so later edits cannot disturb orders already running.

2. Factory Method

Problem: Every customer order is its own workflow instance and needs fresh task objects (Breakfast, Lunch, Dinner, Packaging) created from the shared definition. A change to Customer A's order must never affect Customer B's.

Participants:

GoF Participants	Class Names
Creator	WorkItemFactory
ConcreteCreator	BreakfastTaskFactory, LunchTaskFactory, DinnerTaskFactory, PackagingTaskFactory, FullDayPackageFactory
Product	WorkElement
ConcreteProduct	BreakfastTask, LunchTask, DinnerTask, PackagingTask, FullDayPackage

Collaboration: When an instance starts, the WorkflowEngine goes through each work-item spec in the WorkflowDefinition and hands it to the matching WorkItemFactory. The factory does the shared setup, such as linking the new element to the instance, while its concrete subclass decides which element to create, for example a BreakfastTask or a FullDayPackage. The engine only receives a WorkElement back and never names a concrete class.

Variation: The kinds of work an instance contains differ between processes (meal tasks here, laboratory tests or approvals elsewhere), and new kinds appear over time. Only the creator hierarchy has to grow when this happens.

Why is it appropriate: The factory is the instance-creation side of the definition/instance split. It gives every customer order its own freshly built objects from the shared definition, so changing one customer's address or cancelling their order cannot affect another order.

3. Composite

Problem: Orders can be a single meal or a full-day bundle, and a bundle's packaging may start only when all selected meals are ready. The engine should treat a meal and a bundle uniformly without type checks.

Participants:

GoF Participants	Class Names
Component	WorkElement
Leaf	Task (BreakfastTask, LunchTask, DinnerTask, PackagingTask)
Composite	WorkGroup (FullDayPackage)
Client	WorkflowEngine

Collaboration: The WorkflowEngine treats every WorkElement the same way and does not know whether it is a single Task or a WorkGroup. A FullDayPackage passes the engine's request on to its meal tasks, and it counts as complete only when all of them are complete. The PackagingTask depends on that, so it only becomes available after every meal in the bundle is ready.

Variation: The shape of an order varies from a single meal to a bundle of up to three meals. In other workflows, work can nest more deeply as stages and sub-processes with sequential or parallel children.

Why is it appropriate: The Composite represents the nested work structure of a workflow. It lets the engine start, track and complete one meal or a whole bundle through one interface, so the packaging dependency does not need type checks or if/else chains.

4. State

Problem: An order's valid actions depend on where it is. It can't be cancelled once in transit, and it can't be advanced to "In transit" before packaging is done.

Participants:

GoF Participants	Class Names
State	InstanceState
ConcreteState	CreatedState, PreparingState, PackagingState, InTransitState, DeliveredState, CancelledState, FailedState
Context	WorkflowInstance

Collaboration: The WorkflowInstance hands every request it receives to its current InstanceState and does not decide anything itself. Each ConcreteState either carries out the request and switches the instance to the next state, such as PreparingState moving the order on to PackagingState, or refuses it, such as InTransitState refusing a cancellation.

Variation: The statuses an order passes through, and the actions each one permits, differ between processes. A workflow with an approval stage, for instance, would add states and change which transitions are valid.

Why is it appropriate: The State pattern gives the running instance a controlled lifecycle. Every operation is checked by the current state, so invalid actions such as cancelling an order in transit are rejected in one place instead of through status flags and switch statements.

5. Strategy

Problem: The delivery step must choose one of three couriers based on the customer's area, and other organisations would assign work by role or workload instead.

Participants:

GoF Participants	Class Names
Strategy	AssignmentStrategy
ConcreteStrategy	RoleBasedAssignment, WorkloadBasedAssignment, RegionBasedAssignment
Context	AssignmentServi1.bce

Collaboration: When a work item becomes available, the AssignmentService asks the AssignmentStrategy named in the definition to choose a participant from the candidates. RegionBasedAssignment looks at the order's delivery area and picks the courier covering it, and the AssignmentService then gives the item to that courier. Another workflow could name a different strategy without the AssignmentService changing.

Variation: How work is assigned differs between organisations and even between work items, whether by role, by workload or by a rule on workflow data such as delivery area. Each of these rules is a separate algorithm that can be swapped without touching the tasks.

Why is it appropriate: The Strategy is the routing and assignment decision point of the system. It keeps the question "who does this work" out of the task classes, so changing the rule, or choosing a different courier-selection approach, does not require rewriting any work item.

6. Chain of Responsibility

Problem: When ingredients run out or a delivery fails, the issue must climb from kitchen staff to supervisor to branch manager until someone resolves it, and nobody should need to know who handles what.

Participants:

GoF Participants	Class Names
Handler	EscalationHandler
ConcreteHandler	KitchenStaffHandler, SupervisorHandler, BranchManagerHandler
Client	WorkflowEngine

Collaboration: When a task reports an Issue, the WorkflowEngine passes it to the first EscalationHandler in the chain, KitchenStaffHandler. Each handler either resolves the issue or passes it to the next one (SupervisorHandler, then BranchManagerHandler). If nobody resolves it, the engine marks the instance as failed.

Variation: The number of escalation levels, their order, and which kinds of issue each level can resolve differ between organisations and between types of problem.

Why is it appropriate: This pattern is the exception-handling side of the system. The code that raises an issue does not need to know who resolves it. The engine only needs to know the first handler in the chain.

7. Observer

Problem: When packaging finishes, several unrelated things must happen: customer support is alerted, an SMS goes out, the audit log is written, and the next step is made available. The core model shouldn't know about any of them.

Participants:

GoF Participants	Class Names
Subject	WorkflowEventPublisher
ConcreteSubject	WorkflowInstance
Observer	WorkflowListener
ConcreteObserver	AuditLogger, NextItemActivator, SmsNotifier, SupportDashboardListener

Collaboration: The listeners register with the WorkflowInstance when it is created. When a task finishes, the instance publishes an event to all of them without knowing what they do. SmsNotifier messages the customer, SupportDashboardListener alerts customer support, AuditLogger adds the event to the history, and NextItemActivator makes the next work item available, which can also move the instance on to its next state. The listeners work independently and do not know about each other.

Variation: Which parts of the system care about a given event, and what they do in response, differ between workflows and organisations. The same event might trigger an SMS today and an email or a warehouse update tomorrow.

Why is it appropriate: The Observer decouples the core model from everything that reacts to it. The model publishes events without knowing who receives them, so a new notification or integration only needs a new listener to be attached.

8. Adapter

Problem: The three courier companies have incompatible systems that TIVIDY did not design, but a delivery task must work with any of them.

Participants:

GoF Participants	Class Names
Adaptee	CourierAApi, CourierBApi, CourierCApi
Target	ExternalServiceConnector
Adapter	CourierAAdapter, CourierBAdapter, CourierCAdapter
Client	DeliveryTask

Collaboration: The DeliveryTask works only with the ExternalServiceConnector interface, using the courier chosen by the Strategy. The matching CourierAdapter holds that courier's API, translates TIVIDY's request into the courier's own format, and converts the courier's response back into something TIVIDY understands.

Variation: Each courier exposes a different interface, data format and status vocabulary, and the set of external systems can grow or change over time.

Why is it appropriate: The Adapter is the integration boundary of the system. The engine and DeliveryTask depend only on ExternalServiceConnector, so foreign systems stay outside the core and adding a fourth courier means writing one new adapter.

9. Decorator

Problem: A work element sometimes needs extra behaviour wrapped around it, such as applying a discount coupon to the order cost, writing audit entries or raising its priority. These extras must be combinable in any order without a separate subclass for every combination, and without making the task a different kind of task.

Participants:

GoF Participants	Class Names

Component	WorkElement
ConcreteComponent	Task (BreakfastTask, LunchTask, DinnerTask, PackagingTask), FullDayPackage
Decorator	WorkElementDecorator
ConcreteDecorator	CouponDecorator, AuditedDecorator, PriorityDecorator

Collaboration: Each decorator shares the WorkElement interface and wraps another WorkElement. For example, CouponDecorator wraps a FullDayPackage and lowers the order cost by the coupon rule before the cost reaches the engine. Decorators can be wrapped around each other, such as AuditedDecorator around CouponDecorator around the bundle, and each passes everything else straight through to the element inside it.

Variation: Which extras apply to an item (one or several coupons, auditing, priority) differs per order and per workflow.

Why is it appropriate: The Decorator lets cross-cutting behaviour be attached to tasks and bundles without modifying them or creating a subclass per combination.

10. Memento

Problem: If the customer cancels or changes a coupon or address, the order must return to its previous state cleanly, and the history of what happened must not be erased.

Participants:

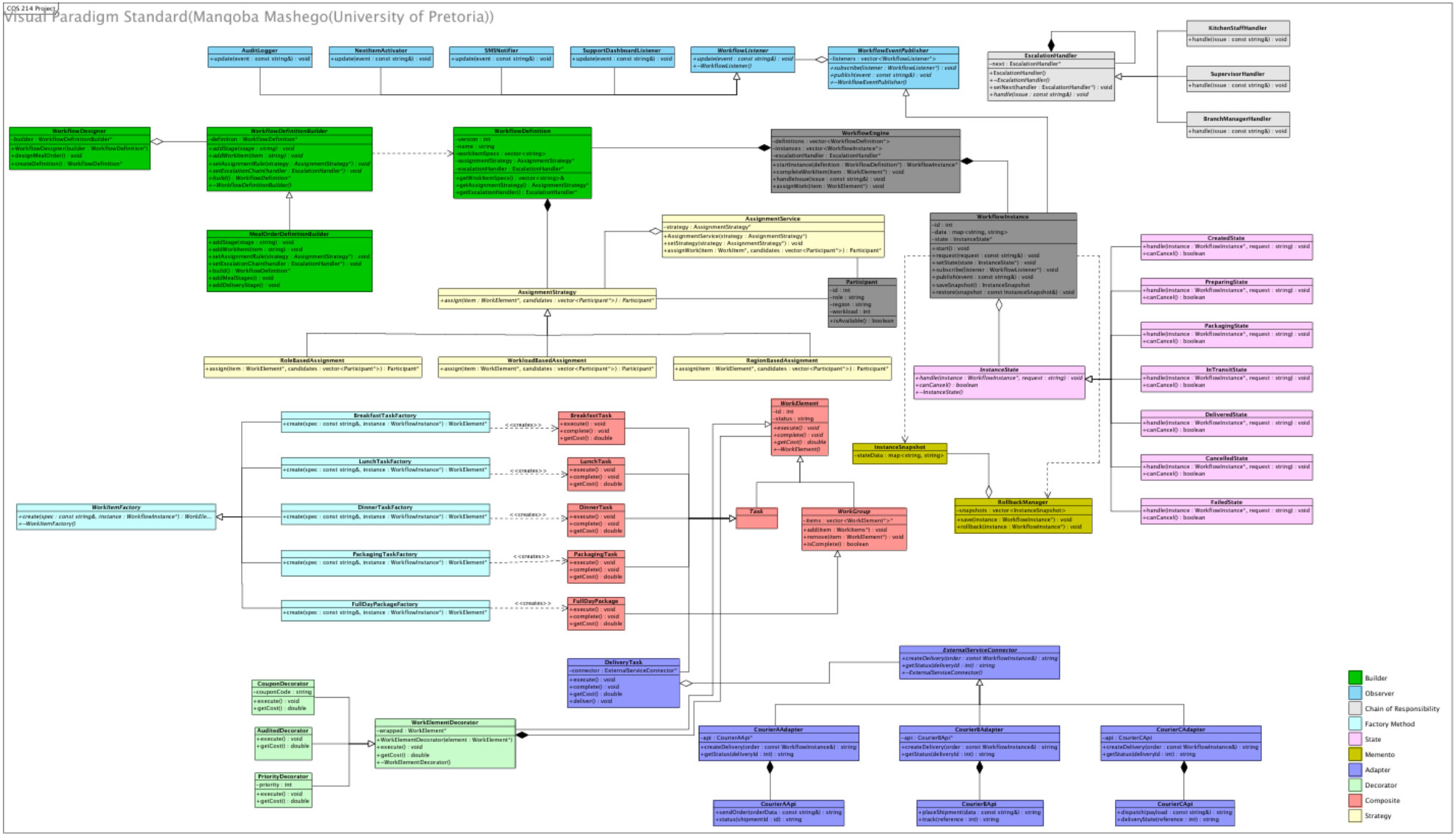
GoF Participants	**Class Names**
Originator	WorkflowInstance
Memento	InstanceSnapshot
Caretaker	RollbackManager

Collaboration: Before a risky change, the RollbackManager asks the WorkflowInstance to save a copy of its current state, and it stores the resulting InstanceSnapshot without reading it. On rollback, the manager gives the latest snapshot back to the instance, which restores itself from it. The AuditLogger then records the rollback, so the history only grows.

Variation: What needs to be captured (payload, states, assignments) and how many levels of undo are kept can differ between workflows.

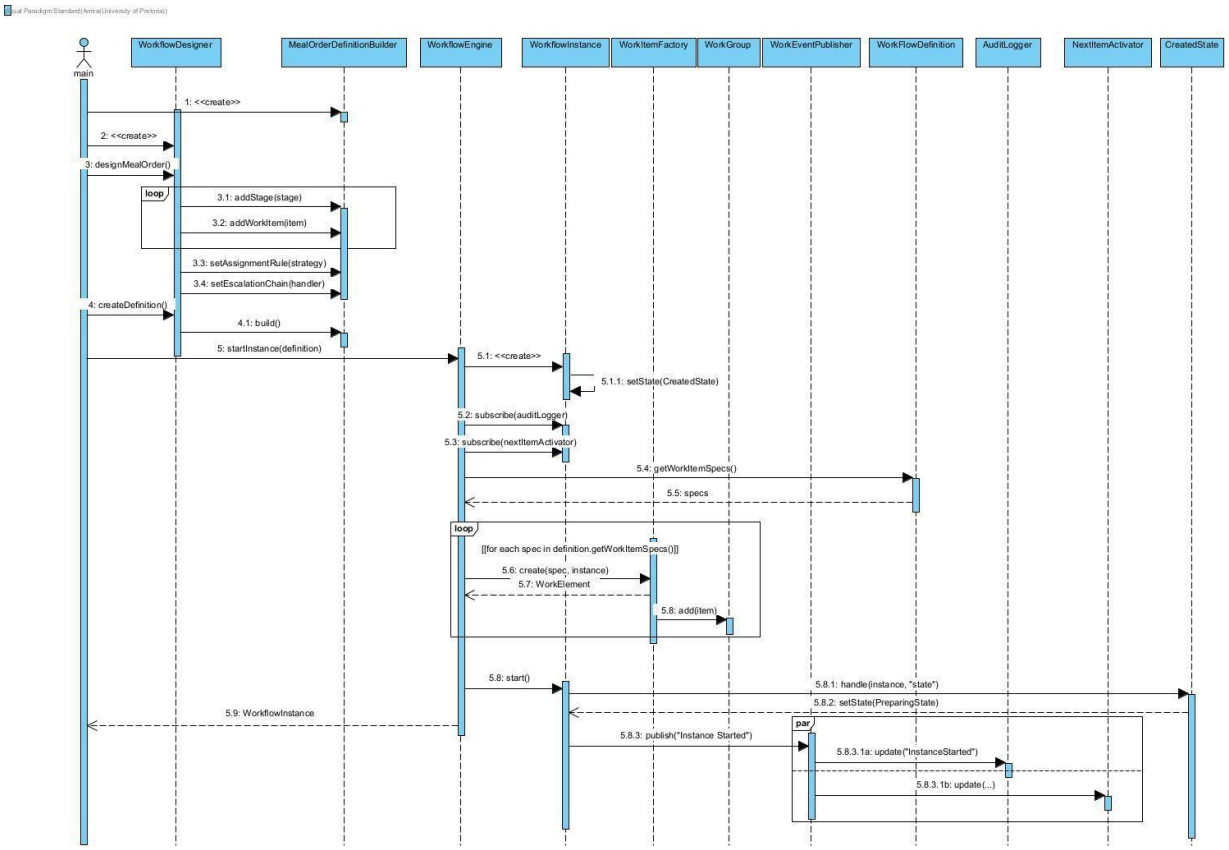
Why is it appropriate: The Memento is the undo and rollback side of the system. Restoring from a snapshot returns the order to a known-good point while the history only grows, so the record stays a

Task 5: UML ClassDiagram

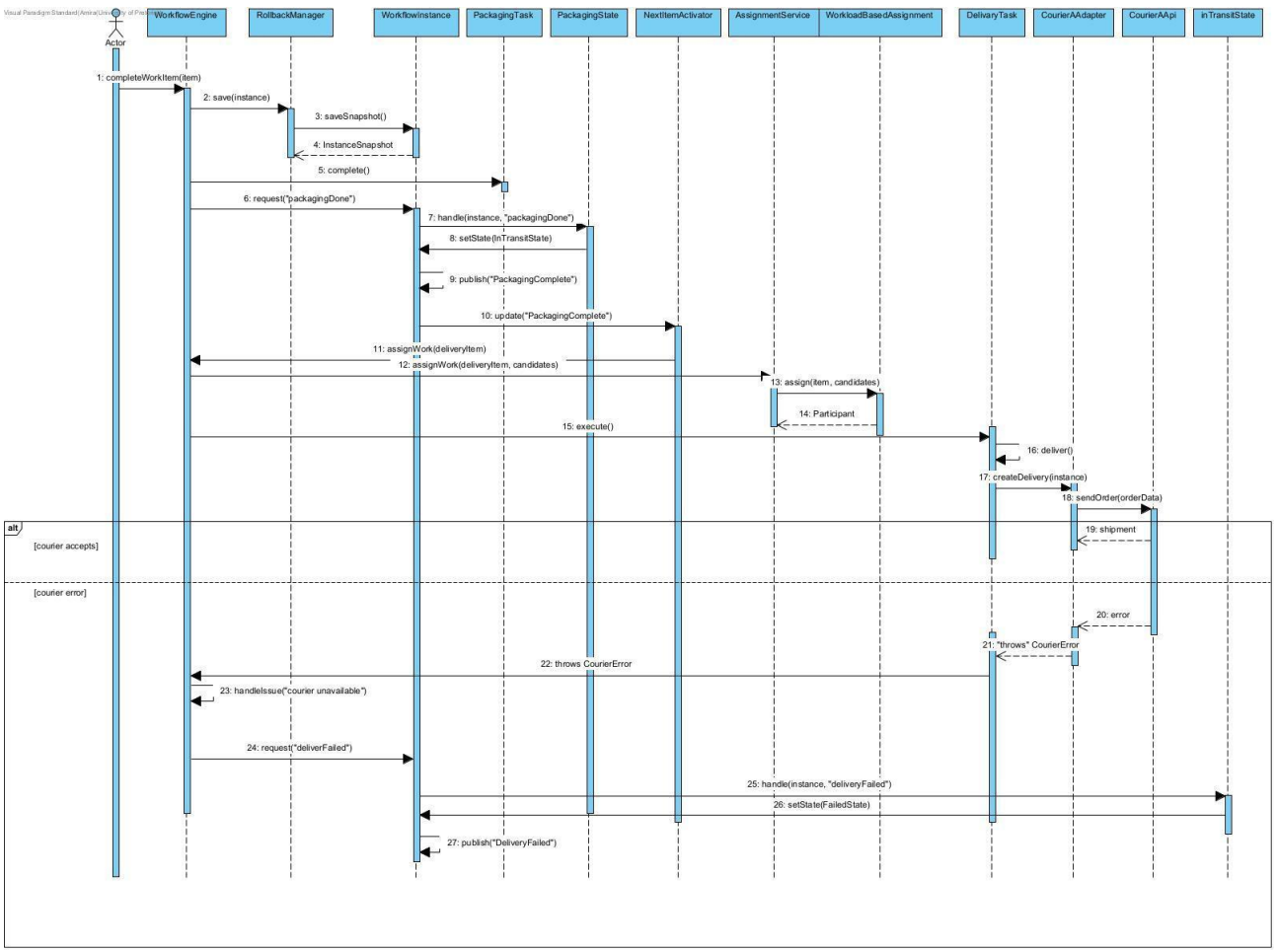


Task 6: Runtime Behaviour Diagrams

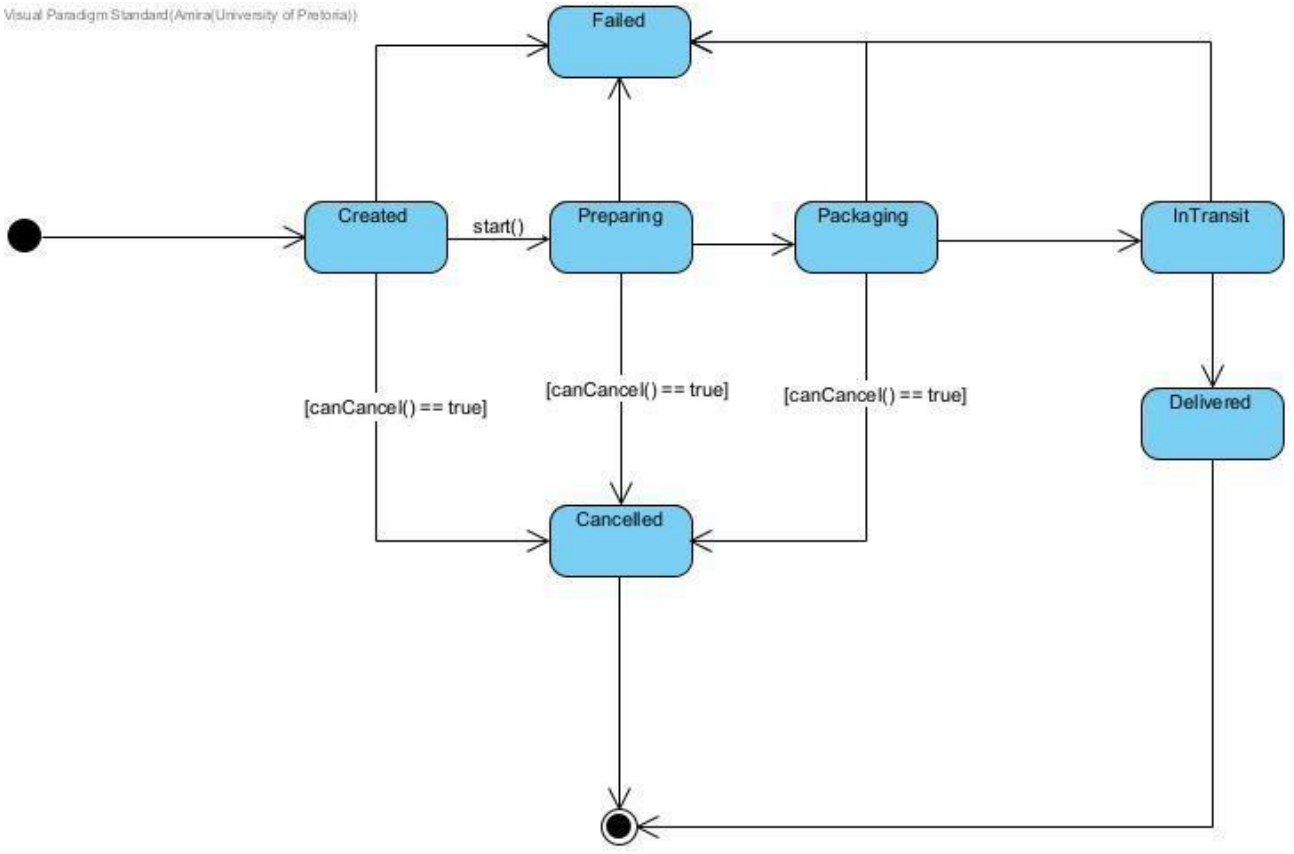
UML Sequence Diagram: Creating and starting a workflow instance



UML Sequence Diagram: Completing a work item, assigning the next one, and handling a courier failure



UML State Diagram: Workflow instance lifecycle



Task 7

- **The problem or question being considered:**
    - Whether the name that we have given to our uml class diagram is too specific to the scenario rather than that of a general workflow system that should be able to work for multiple systems.

- **the original design or assumption:**
    - A few of our initial class names used vocabulary closely linked to our chosen scenario that will be used to test our workflow. Such as kitchenstaff manager, branch manager and other similar role specified classes.

- **the change that was made:**

    - Changes have not been made yet as it is something we will discuss with our tutor during consultation. Once it has been discussed and we have decided as a group we will then make the relevant updates to the required diagrams/sections.

- **why the change is likely to be made:**
    - To make our workflow system fit a more general sense so that it can be reused for multiple different purposes. This also helps to improve our level of abstraction.

- **updates to the relevant diagrams:**

    - None at this current moment

Contribution Statement

For this deliverable, tasks were allocated based on what each member felt was their strength and/or interest. The work was mostly divided by tasks with members then pushing the work from their branches to the main as each task finished allowing us to work in parallel. Once a member finished a section of the specification, they would notify the rest of us as teammates to verify what had been submitted and allow each other the opportunity to offer our opinions on anything that we felt needed to change. All members reviewed the integrated design to ensure the diagrams and documentation told a coherent story. The task allocation was as follows:

- **Task 1 :** Giselle
- **Task 2:** Nontokozo
- **Task 3:** Shepherd
- **Task 4:** Ngeletshedzo
- **Task 5:** Manqoba
- **Task 6:** Amira
- **Task 7/ Pdf:** Nare

References

Camunda, n.d. *Versioning process definitions.* [Online]
Available at: https://docs.camunda.io/docs/components/best-practices/operations/versioning-process-definitions/
[Accessed 04 10 2026].
Object Management Group, 2011. *Business Process Model and Notation (BPMN).* [Online]
Available at: https://www.omg.org/spec/BPMN/2.0/PDF
[Accessed 04 10 2026].
Russell, N., van der Aalst, W., ter Hofstede, A. & Edmond, D., 2005. *Workflow resource patterns: identification, representation and tool support.* Berlin, Springer.
van der Aalst, W., 2013. Business process management: a comprehensive survey. *ISRN Software Engineering,* pp. 1-37.
van der Aalst, W., ter Hofstede, A., Kiepuszewski, B. & Barros, A., 2003. Workflow patterns. *Distributed and Parallel Databases,* 14(1), pp. 5-51.
Workflow Management Coalition, 1994. *The Workflow Reference Model.* [Online]
Available at: https://www.aiai.ed.ac.uk/project/wfmc/ARCHIVE/DOCS/refmodel/rmv1-16.html
[Accessed 04 10 2026].